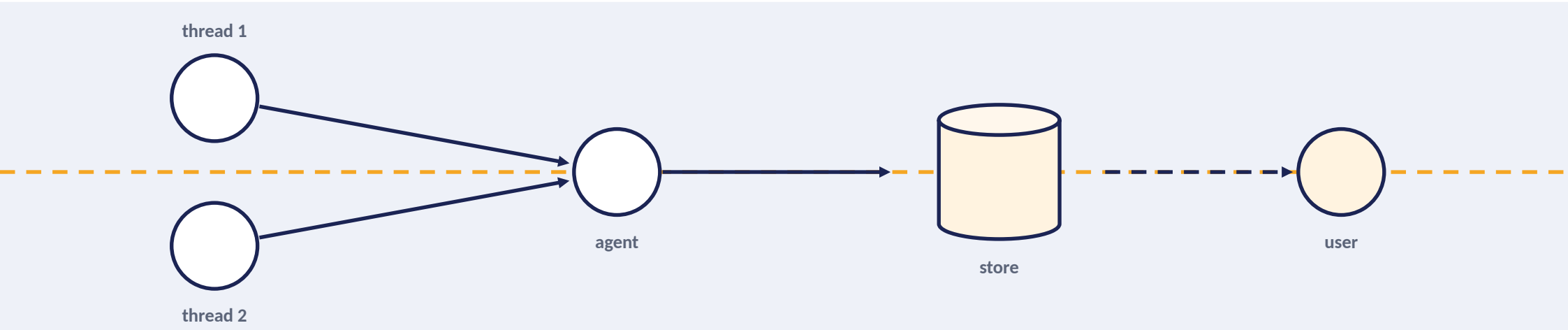
LANGGRAPH · CHAPTER 11 · FLASHCARDS

Semantic, Episodic & Procedural Memory

Facts, examples, rules and skills

11 code cards on semantic, episodic and procedural memory



A fact, an example, or a rule?

Name what the agent is missing before you choose where to keep it

- All three share one Store: the namespace's last part separates them, the user_id isolates users.
- Semantic is written on the hot path, mid-answer; episodic and procedural in the background, after it.

TEXT

```
              SEMANTIC              EPISODIC                PROCEDURAL
stores        facts                 past cases + outcome    instructions
example       "prefers Python"      an approved             "under 100 words"
                                    recursion answer
suffix        "facts"               "episodes"              "instructions"
key           one per fact          a new UUID each         a fixed section name
found by      embedding search      embedding search        exact get()
written by    the agent (a tool)    the app, after a        an optimizer,
                                    human approves          reviewed first
reaches the   system-prompt         few-shot examples       system-prompt
model as      facts                 beside the request      sections

namespace = ("assistant", user_id, suffix)
skill = a written method kept as a file, identical across users
```

The agent saves facts with a tool

LangMem's manage_memory lets the model store a fact the moment it hears one

- The index makes the Store embed each value, so search(query=...) finds memories by meaning.
- {langgraph_user_id} is filled from config on every call: one tool, a separate folder per user.

PYTHON

```python
from langgraph.store.memory import InMemoryStore
from langmem import create_manage_memory_tool

# The index embeds every saved value: search(query=...) ranks by meaning
store = InMemoryStore(
    index={"embed": "openai:text-embedding-3-small", "dims": 1536})

# Filled from config["configurable"]["langgraph_user_id"] on each call
FACTS_NAMESPACE = ("assistant", "{langgraph_user_id}", "facts")
manage_memory = create_manage_memory_tool(namespace=FACTS_NAMESPACE)

# What the model sees: manage_memory(content, action, id)
#   action: "create" (the default) | "update" | "delete"

# Inside a run with config {"configurable": {"langgraph_user_id": "maya"}}:
manage_memory.invoke({"content": "prefers examples in Python",
                      "action": "create"})
# 'created memory 6353...'  -> saved under ("assistant", "maya", "facts")
```

SEMANTIC · READING

Your code reads before every model call

A @dynamic_prompt middleware searches the Store first, so the lookup is never skipped

- With a search_memory tool, looking was optional: in a new chat the model never looked.
- The returned string becomes that call's system prompt: the model doesn't choose to look; it always sees.

PYTHON

```python
@dynamic_prompt
def recall_facts(request: ModelRequest) -> str:
    """Runs before every model call: read memory in code, never skipped."""
    user_id = get_config()["configurable"]["langgraph_user_id"]
    question = next((m.content for m in reversed(request.messages)
                     if m.type == "human"), "")
    found = store.search(("assistant", user_id, "facts"),
                         query=question, limit=5)
    lines = [f"- {item.value['content']}" for item in found]
    facts = "\n".join(lines) or "- (nothing saved yet)"
    return f"{SYSTEM_PROMPT}\n\nWhat you know about this user:\n{facts}"

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[create_manage_memory_tool(namespace=FACTS_NAMESPACE)],  # writing
    middleware=[recall_facts],          # reading: your code, before each call
    store=store,                        # every run must share this Store
)
```

SEMANTIC · TWO RUNS

Two runs, one Store, no shared messages

Run 2 knows about the interviews only because recall_facts looked them up

- In run 1 the middleware runs before both model calls: 0 facts, then the 1 fact just saved.
- Inspect the Store whenever an agent can write memory; another user's namespace stays empty.

PYTHON

```python
config = {"configurable": {"langgraph_user_id": "maya"}}

first = ("I'm preparing for data-engineering interviews, "
         "and I prefer examples in Python.")
agent.invoke({"messages": [{"role": "user", "content": first}]}, config=config)
#   [memory] recalled 0 fact(s) for maya
#   AI: tool_calls -> manage_memory(content="...interviews... Python...")
#   [memory] recalled 1 fact(s) for maya
#   AI: Noted! ...

second = "Give me one practice question."      # a brand-new conversation
agent.invoke({"messages": [{"role": "user", "content": second}]}, config=config)
#   [memory] recalled 1 fact(s) for maya
#   AI: a data-engineering question with a Python example (no tool call)

store.search(("assistant", "maya", "facts"))           # what the agent saved
store.search(("assistant", "another-user", "facts"))   # []: isolated
```

An approved answer becomes an episode

Your code saves the request and the human-approved answer; the model never writes episodes

- Only approved answers become examples, so a mediocre answer can't reinforce itself.
- A fresh UUID per key: episodes pile up like case files, unlike a profile that gets overwritten.

PYTHON

```python
import uuid

def episodes_namespace(user_id: str) -> tuple[str, str, str]:
    return ("assistant", user_id, "episodes")

def save_episode(user_id: str, request: str, approved_answer: str) -> None:
    """Save one human-approved exchange. The model never writes episodes."""
    store.put(
        episodes_namespace(user_id),
        str(uuid.uuid4()),        # episodes accumulate; each gets its own key
        {"request": request, "answer": approved_answer,
         "source": "human_approved"},
    )

# Maya edited the long answer into: analogy + 5-line example + one mistake
save_episode("maya", "Explain recursion.", APPROVED_RECURSION_ANSWER)
# 1 episode under ("assistant", "maya", "episodes")
```

EPISODIC · FEW-SHOT

Similar past cases go into the prompt

The node searches the user's episodes with the new request and shows the closest as examples

- The node asks for config and store: BaseStore; LangGraph injects the Store given to compile().
- The instructions never change between runs; only the block of approved examples does.

PYTHON

```python
def answer(state: State, config: RunnableConfig, store: BaseStore) -> dict:
    user_id = config["configurable"]["langgraph_user_id"]
    # The application, not the model, decides which past episodes to show
    matches = store.search(episodes_namespace(user_id),
                           query=state["request"], limit=2)
    system = (
        "You explain programming concepts.\n\n"
        f"{format_examples(matches)}\n\n"    # "Request: ... Approved: ..."
        "If approved answers are shown, follow their structure and length "
        "closely, but write new content for the new request."
    )
    reply = model.invoke([{"role": "system", "content": system},
                          {"role": "user", "content": state["request"]}])
    return {"answer": reply.content}

graph = builder.compile(store=store)      # START -> answer -> END

graph.invoke({"request": "Explain dynamic programming."}, config=maya)
# 1 episode retrieved -> analogy, short Python example, one mistake
# as "another-user": 0 episodes -> the generic style again
```

Instructions kept as named sections

The system prompt is rebuilt from stored sections on every call, each read by exact key

- Stable section names mean plain get() calls: no index, no embeddings, no wrong match.
- Seed a default only when its key is missing, or every restart erases what was learned.

PYTHON

```python
store = InMemoryStore()              # no index: sections are read by exact key
DEFAULT_SECTIONS = {
    "tone": "Be friendly and encouraging.",
    "answer_format": "Explain the concept thoroughly with background and ...",
}

def seed_defaults(user_id: str) -> None:     # never overwrite learned text
    for key, text in DEFAULT_SECTIONS.items():
        if store.get(instructions_namespace(user_id), key) is None:
            store.put(instructions_namespace(user_id), key,
                      {"prompt": text, "source": "default"})

def read_section(user_id: str, key: str) -> str:
    return store.get(instructions_namespace(user_id), key).value["prompt"]

def build_system_prompt(user_id: str) -> str:
    seed_defaults(user_id)           # rebuilt on every call, never stored whole
    return ("You are a study assistant.\n"
            f"Tone: {read_section(user_id, 'tone')}\n"
            f"Answer format: {read_section(user_id, 'answer_format')}")
```

Feedback rewrites one section

A separate optimizer call turns the trajectory plus feedback into revised sections

- when_to_update keeps feedback about length from rewriting the tone section.
- Save only the sections that changed, tagged with their source, and review each one before the put.

PYTHON

```python
optimizer = create_multi_prompt_optimizer("openai:gpt-4o-mini", kind="prompt_memory")

def learn_from_feedback(user_id, question, answer, feedback) -> list[str]:
    prompts = [
        {"name": name,
         "prompt": read_section(user_id, spec["key"]),
         "update_instructions": "Keep each instruction short and specific.",
         "when_to_update": spec["when_to_update"]}
        for name, spec in SECTION_SPECS.items()]
    trajectory = [HumanMessage(question), AIMessage(answer)]
    proposed = optimizer.invoke({"trajectories": [(trajectory, feedback)],
                                 "prompts": prompts})
    updated = []
    for old, new in zip(prompts, proposed):      # returned in the same order
        if new["prompt"] != old["prompt"]:       # skip unchanged sections
            key = SECTION_SPECS[old["name"]]["key"]
            store.put(instructions_namespace(user_id), key,
                      {"prompt": new["prompt"],
                       "source": "optimized_from_feedback"})
            updated.append(old["name"])
    return updated     # e.g. ['answer-format'] after "Too long. Keep answers ..."
```

A method in SKILL.md, loaded by a node

One node loads the skill into state; it reaches the model only when the next node adds it

- Loading a skill is an ordinary node: read a file and return a partial state update.
- State isn't context: leave the skill out of the messages and the run still succeeds, minus the method.

PYTHON

```python
# skills/paper_review/SKILL.md opens with front matter:
#   ---
#   name: paper-review
#   description: Review an academic research paper ... Use when asked to ...
#   ---

def load_skill(state: ReviewState) -> dict:
    skill_path = HERE / "skills" / "paper_review" / "SKILL.md"
    return {"skill_instructions": skill_path.read_text(encoding="utf-8")}

def review_paper(state: ReviewState) -> dict:
    # The skill reaches the model only because this line adds it
    system = (SYSTEM_PROMPT + "\n\nSpecialized instructions:\n"
              + state["skill_instructions"])
    reply = model.invoke([SystemMessage(system), HumanMessage(state["paper"])])
    return {"review": reply.content}

# START -> load_skill -> review_paper -> END
# [skill] loaded skills/paper_review/SKILL.md (184 words)
# ## 1. Summary ... ## 7. Recommendation  Weak Accept. ...
```

A catalog in the prompt, a load_skill tool

The agent always sees the skill names; a skill's full text enters only when it asks

- The model picks a name, never a path: an unknown name (or "../../.env") gets an error, not a file.
- InjectedState passes the run's state into the tool and hides that parameter from the model.

PYTHON

```python
CATALOG = {}                        # built at startup: names + descriptions only
for skill_file in sorted(SKILLS_DIR.glob("*/SKILL.md")):
    fields, _ = read_skill_file(skill_file)
    CATALOG[fields["name"]] = {"description": fields["description"],
                               "path": skill_file}

@tool
def load_skill(name: str, state: Annotated[dict, InjectedState]) -> str:
    """Load the full instructions for one skill from the catalog ..."""
    if name not in CATALOG:                          # only catalog entries open
        return f"Unknown skill '{name}'. Available: {', '.join(CATALOG)}."
    loaded_marker = f"Skill '{name}' is now loaded."
    if any(m.type == "tool" and m.content.startswith(loaded_marker)
           for m in state["messages"]):              # never send it twice
        return f"Skill '{name}' is already loaded above. Apply it now."
    _, body = read_skill_file(CATALOG[name]["path"])
    return f"{loaded_marker} Don't load it again; apply it.\n\n{body}"

agent = create_agent(model=model, tools=[load_skill],
                     system_prompt=SYSTEM_PROMPT)    # the prompt lists CATALOG
# 75 words of catalog in the prompt, instead of 439 words of methods
```

Memory that misleads quietly

Semantic search always returns its closest match, and a careless seed erases learned rules

- search(query=...) has no relevance cutoff: an unrelated query still returns the nearest fact.
- Read named sections with get(), and put() a default only when its key is missing.

PYTHON

```python
ns = ("assistant", "maya", "facts")
store.put(ns, "f1", {"content": "prefers examples in Python"})
store.search(ns, query="zebra quantum banana", limit=5)
# [Item(key='f1', score=0.17, ...)]   <- returned anyway: no cutoff

# Broken seed (Exercise 11.6): the "is None" check is gone
def seed_defaults(user_id: str) -> None:
    for key, text in DEFAULT_SECTIONS.items():
        store.put(instructions_namespace(user_id), key,
                  {"prompt": text, "source": "default"})
# build_system_prompt() seeds on every call, so the learned
# answer_format is overwritten the next time the assistant answers

# Checked on langgraph 1.2.6 / langmem 0.0.30, with a stand-in embedder
```